

Information retrieval

The traditional approach to IR is based on
MANNING, C., RAGHAVAN, P., SCHÜTZE, H.
Introduction to Information Retrieval. New York:
Cambridge University Press, 2008.

Goals

- to *retrieve* texts that are *relevant* to an *information need*
 - match a user query (=information need)
 - with predicted usefulness of the texts
- two main aims
 - coverage (recall) – to present most of the relevant texts
 - precision – to not present most of the non-relevant texts

IR scale

- personal search
 - searching in a personal computer
- enterprise, institutional, domain specific search
 - searching in internal databases, patent databases, research articles from a domain
- web search
 - browsing billions of documents stored on millions of computers
 - large scale, working with hyperlinks, ...

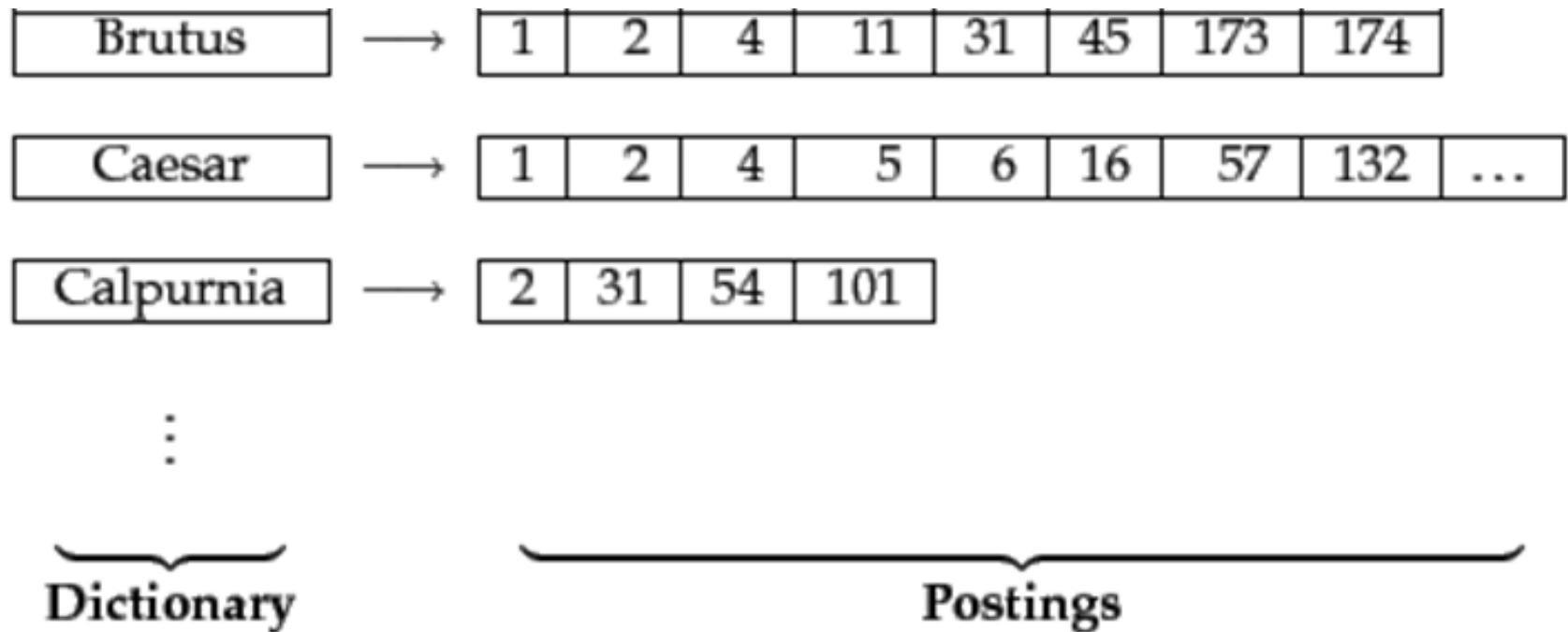
Basic IR

- the simplest way is to linearly read all documents and find desired words = grepping
- this simple approach has several limitations
 - might be slow (for large data volumes)
 - is not very flexible (e.g., searching “Romans NEAR countrymen” where NEAR might be defined as ‘within 5 words’ or within the same sentence)
 - ranking is not possible (the best answer is usually expected to be first)

Inverted index

- term-document matrices for large collections
 - cannot fit into computer memory
 - are very sparse (it is better to record only things that do occur)
- inverted index
 - we keep the dictionary (vocabulary, lexicon) and the list of term occurrences in documents

Inverted index



Inverted index

- to improve efficiency

- dictionary items might also contain document frequencies
- posting lists are sorted

term	doc. freq.	→	postings lists
ambitious	1	→	2
be	1	→	2
brutus	2	→	1 → 2
capitol	1	→	1
caesar	2	→	1 → 2
did	1	→	1
enact	1	→	1
hath	1	→	2
I	1	→	1
i'	1	→	1
it	1	→	2
julius	1	→	1
killed	1	→	1
let	1	→	2
	1	→	1

Boolean retrieval

- documents are viewed as sets of words
- terms are combined with the logical operators *and*, *or*, and *not*
- an example: *Brutus AND Calpurnia*
 - locate *Brutus* in the Dictionary
 - retrieve its postings
 - locate *Calpurnia* in the Dictionary
 - retrieve its postings
 - intersect the two postings lists

Intersection of posting lists – an algorithm

```
INTERSECT( $p_1, p_2$ )  
  1   $answer \leftarrow \langle \rangle$   
  2  while  $p_1 \neq \text{NIL}$  and  $p_2 \neq \text{NIL}$   
  3  do if  $docID(p_1) = docID(p_2)$   
  4      then  $\text{ADD}(answer, docID(p_1))$   
  5           $p_1 \leftarrow next(p_1)$   
  6           $p_2 \leftarrow next(p_2)$   
  7      else if  $docID(p_1) < docID(p_2)$   
  8          then  $p_1 \leftarrow next(p_1)$   
  9          else  $p_2 \leftarrow next(p_2)$   
 10 return  $answer$ 
```

Boolean retrieval – optimization

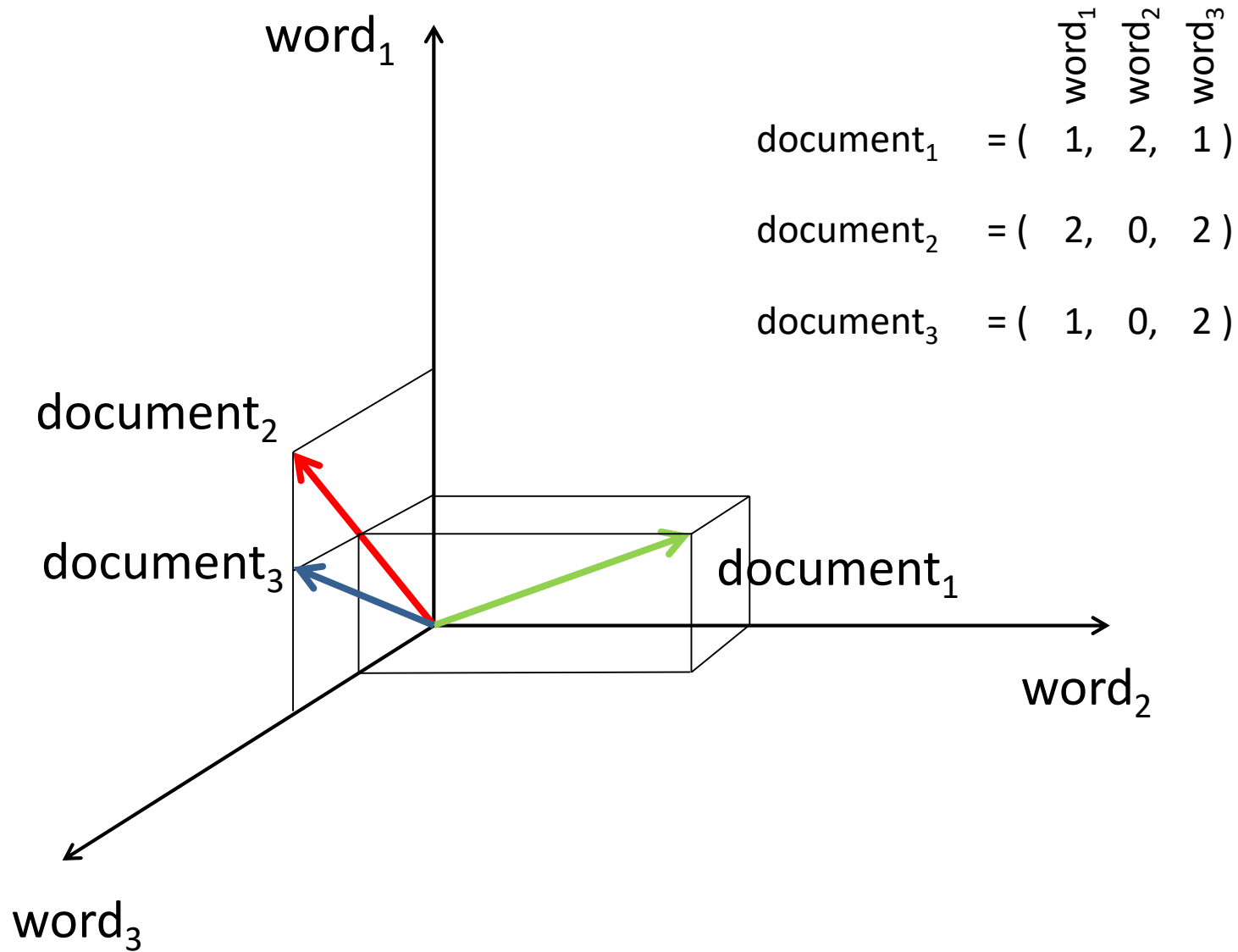
- processing terms in order of increasing document frequency – if we start by intersecting the two smallest postings lists, then all intermediate results must be no bigger than the smallest postings list, and we are therefore likely to do the least amount of total work
- using hash tables for posting lists so they don't have to be traversed linearly
- compression – variable length of terms, variable byte encoding, γ -codes

Boolean retrieval – disadvantages

- *exact match* only
- retrieved documents are *not ranked*
- all terms are equally important
- strict Boolean search is too limited for many information needs
 - using additional operators might help – e.g., wildcards, proximity operators (defining a distance between searched terms), phrase search

Vector space model

- easily enables term relevance *weighting*
- every document is represented by a vector where every term has assigned a weight (very often tf-idf)
- this representation is suitable also for other operations, such as classification or clustering



Factors of term importance (simplified)

- term frequency – the more times a term appears in a document the more important it probably is
- inverse document frequency – common terms are less important than less common terms
- document length – longer documents contain more repeated terms, thus having higher term frequencies

Query as a document

- representing *queries as vectors* in the vector space (might use tf, idf, and normalization)
- documents can be ranked according to their similarity to the query in this space

Shared word count

- a simple measure counting the number of words appearing in both documents
- very intuitive
- can be calculated as the product of two vectors containing zeros and ones representing the documents

Shared word count and bonus

$$\textit{Similarity}(q, D(i)) = \sum_{j=1}^k w(j)$$

$$w(j) = \begin{cases} 1 + \frac{1}{df(j)} & \text{if word } j \text{ occurs in the document and query} \\ 0 & \text{otherwise} \end{cases}$$

$D(i)$ a document from the entire collection

q a query

j a word from the query

k the number of terms in q

$df(j)$ the number of documents where the word j occurs

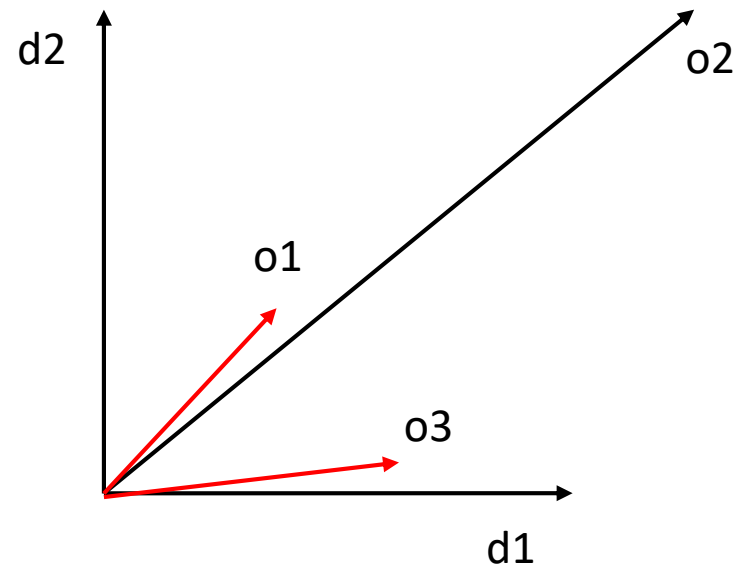
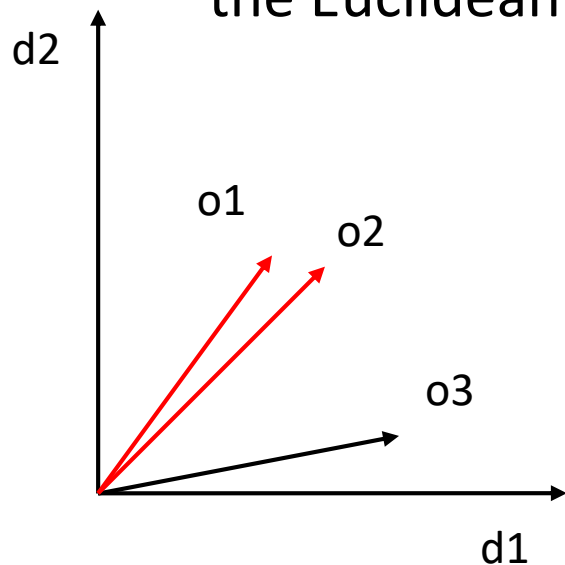
Euclidean distance

- distance between two points in Euclidean space
- good or bad idea?

Euclidean distance

- distance between two points in Euclidean space
- good or bad idea?

Red vectors are most similar to each other when using the Euclidean distance.



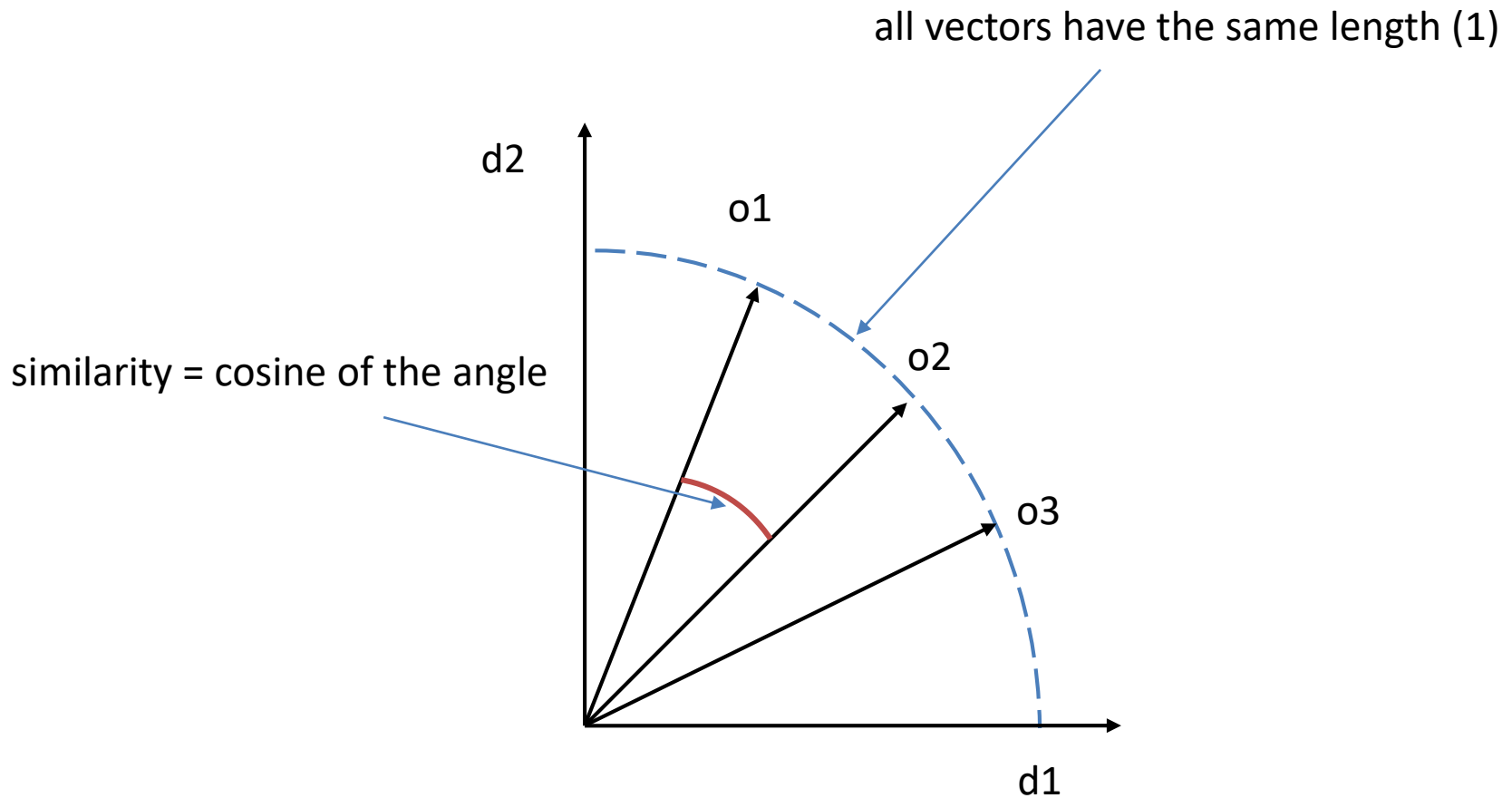
Cosine similarity

- a standard way of quantifying the similarity of two vectors

$$\text{sim}(d_1, d_2) = \frac{d_1 \cdot d_2}{|d_1| * |d_2|} = \frac{\sum_{i=1}^n d_{1i} d_{2i}}{\sqrt{\sum_{i=1}^n d_{1i}^2} \sqrt{\sum_{i=1}^n d_{2i}^2}}$$

- $\text{sim}(d_1, d_2)$ is the cosine of the angle between d_1 and d_2
- the numerator is the dot (inner, scalar) product
- the denominator length-normalizes the vectors d_1 and d_2 to unit vectors (if the vectors are already normalized the similarity can be calculated as $\text{sim}(d_1, d_2) = d_1 \cdot d_2$)

Cosine similarity



Cosine similarity

- values from 0 to 1 (why? – the cosine ranges from -1 to +1)
- the length of vectors is not important
- efficient for calculations, because only nonzero entries are considered
- is not a distance metric ($d(x,y) \geq 0$ – non-negativity, $d(x,y)=0 \Leftrightarrow x=y$ – identity, $d(x,y)=d(y,x)$ – symmetry, $d(x,y) \leq d(x,z)+d(z,y)$ – triangle inequality)
- angular distance might be used instead

$$distance = \frac{2 \cos^{-1}(similarity)}{\pi}$$

Queries as vectors

- having a query q , the score for a document d might be calculated as follows (overlap score)

$$Score(q, d) = \sum_{t \in q} tf \cdot idf_{t,d}$$

- when taking a query as a vector, the similarity with a document d might be calculated using the cosine measure

$$sim(q, d) = \frac{q \cdot d}{|q| * |d|}$$

- top K most similar documents are retrieved

The algorithm

initialize Scores[N] = 0

calculate Lengths[N]

for each query term t

do calculate $w_{t,q}$ and fetch postings list for t

for each pair(d, $tf_{t,d}$) in postings list

do Scores[d] += $wf_{t,d} \times w_{t,q}$

for each d

do Scores[d] = Scores[d]/Lengths[d]

return Top K components of Scores[]

Algorithm modification

- high cost of computing cosine similarities between the query and a large number of documents
- instead, K documents that are likely to be among the K highest scoring documents for a query are retrieved
- such an inexact retrieval is not necessarily, from the user's perspective, a bad thing – the top K documents by the cosine measure are in any case not necessarily the K best for the query
- heuristics
 - find a set A of documents that are contenders, where $K < A \ll N$ (A does not necessarily contain the K top-scoring documents for the query)
 - return the K top-scoring documents in A

Algorithm modification

- index elimination
 - considering only terms with high idf (low idf = stopwords)
 - processing only documents containing many (a special case = all) of the query terms
- champion lists
 - using a pre-computed list of r documents with the highest weights for each term in the dictionary
 - searching only in the union of champion lists for all query terms
- cluster pruning
 - searching the best match between a query and a set of “leaders”, then, searching only in a set of candidates that are in the same cluster as the leader (the documents are clustered first)

Retrieval evaluation

- to measure retrieval effectiveness we need
 - a document collection
 - a set of queries (expressing information needs)
 - a set of relevance judgments (standardly, the values relevant or non-relevant for each query-document pair)
- a retrieval system should be tested against queries that express real information needs (predicted usage of the system)

Retrieval evaluation

- standard measures
 - precision – the fraction of retrieved documents that are relevant

$$P = \frac{\# \text{ relevant items retrieved}}{\# \text{ retrieved items}} = P(\text{relevant}|\text{retrieved})$$

- recall – the fraction of relevant documents that are retrieved

$$R = \frac{\# \text{ relevant items retrieved}}{\# \text{ relevant items}} = P(\text{retrieved}|\text{relevant})$$

Retrieval evaluation

- using the contingency table (confusion matrix)

	relevant	nonrelevant
retrieved	true positives	false positives
not retrieved	false negatives	true negatives

$$P = TP/(TP+FP), R = TP/(TP+FN)$$

- What about accuracy?

$$A = (TP + TN) / (TP + TN + FP + FN)$$

Retrieval evaluation

- a user might prefer
 - having only relevant documents on the top
 - retrieving all / only some relevant documents
- the above-mentioned goals are conflicting
- consider the following queries from the precision and recall perspective:
 - How to tie a bow tie?
 - What do customers write about my company?
 - What special actions do restaurants in Brno have?

Retrieval evaluation

- F-measure

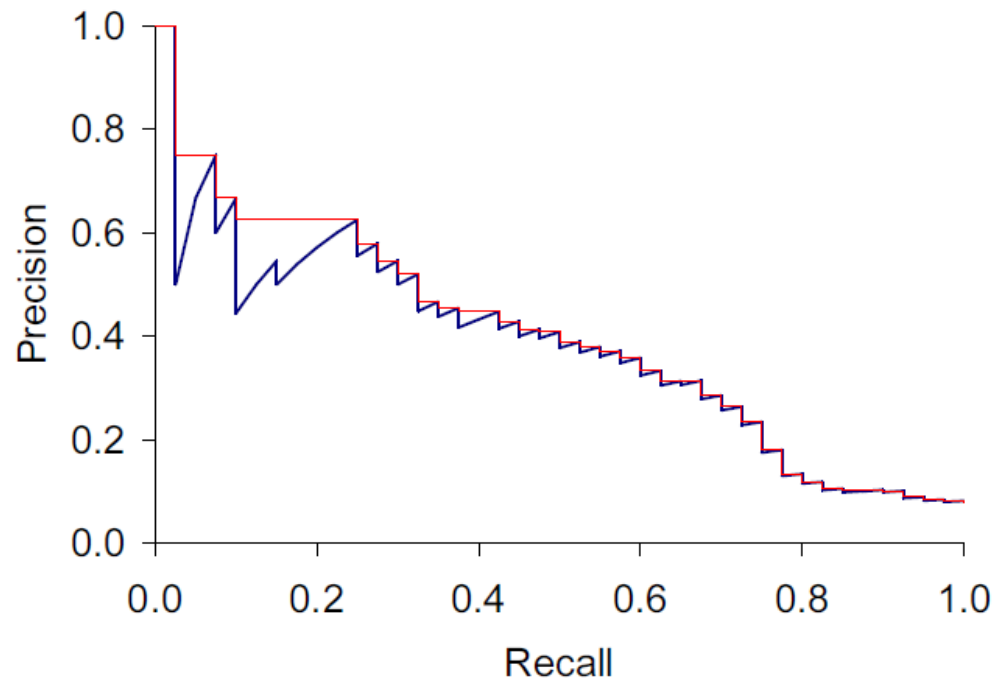
- a trade off between precision and recall
- the weighted harmonic mean of precision and recall

$$F = \frac{1}{\alpha \frac{1}{P} + (1-\alpha) \frac{1}{R}} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}, \text{ where } \beta^2 = \frac{1-\alpha}{\alpha}$$

- balanced – $\alpha=1/2$ or $\beta=1$ ($F_1=2PR/(P+R)$)
- values of $\beta < 1$ emphasize precision, while values of $\beta > 1$ emphasize recall

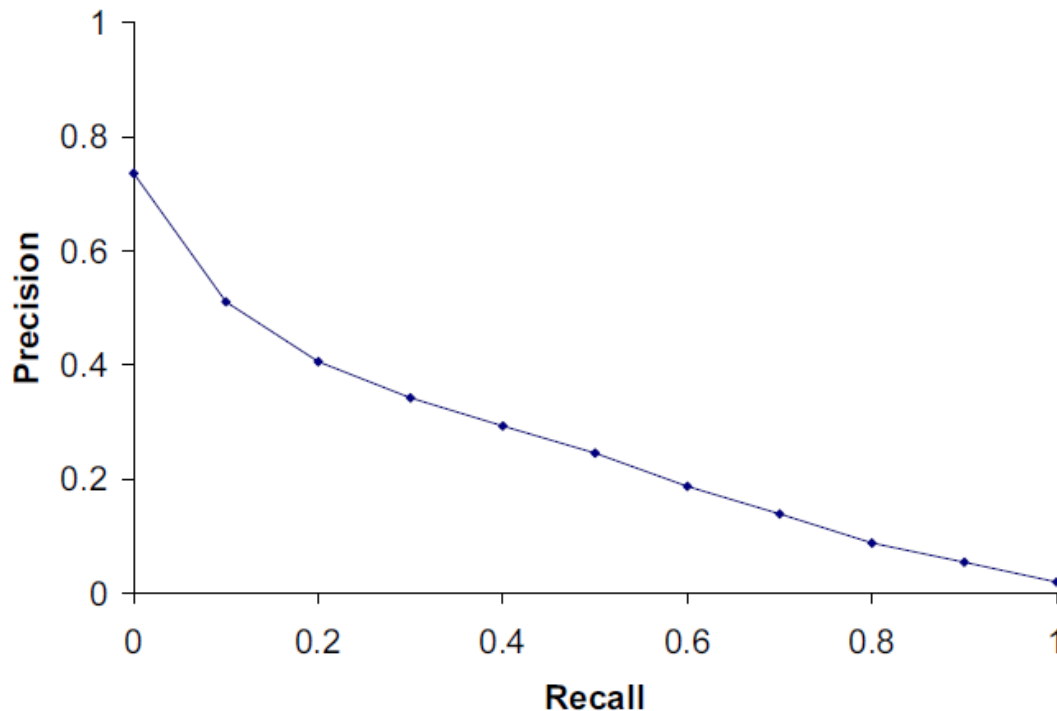
Retrieval evaluation

- Precision/Recall curve
 - blue line – real
 - red line – interpolated



Retrieval evaluation

- 11-point interpolated average precision – measured at the 11 recall levels of 0.0, 0.1, 0.2, . . . , 1.0



Improving search results

- it is often difficult to formulate a query without knowing the collection
- some techniques might help retrieve better results
 - global methods – expand or reformulate query terms independent of the query and results returned
 - local methods – adjust a query relative to the documents that initially appear to match the query

Improving search results

- query expansion
 - new terms are added to the query
 - using thesaurus (a list of synonyms) created by humans, automatically (based on words co-occurrence statistics), or by query log mining

Improving search results

- relevance feedback
 - the user places a query (short, simple)
 - an initial set of results is retrieved
 - the user marks some returned documents as relevant or non-relevant
 - the system computes a better representation of the information need based on the user feedback
 - a revised set of results is retrieved

Neural IR

- neural networks are used in one or more of IR phases
 - query representation
 - document representation
 - relevance assessment (ranking)
- modern deep learning-based NLP techniques enable
 - better understanding the intent (query)
 - better query expansion
 - lower dimensional representations

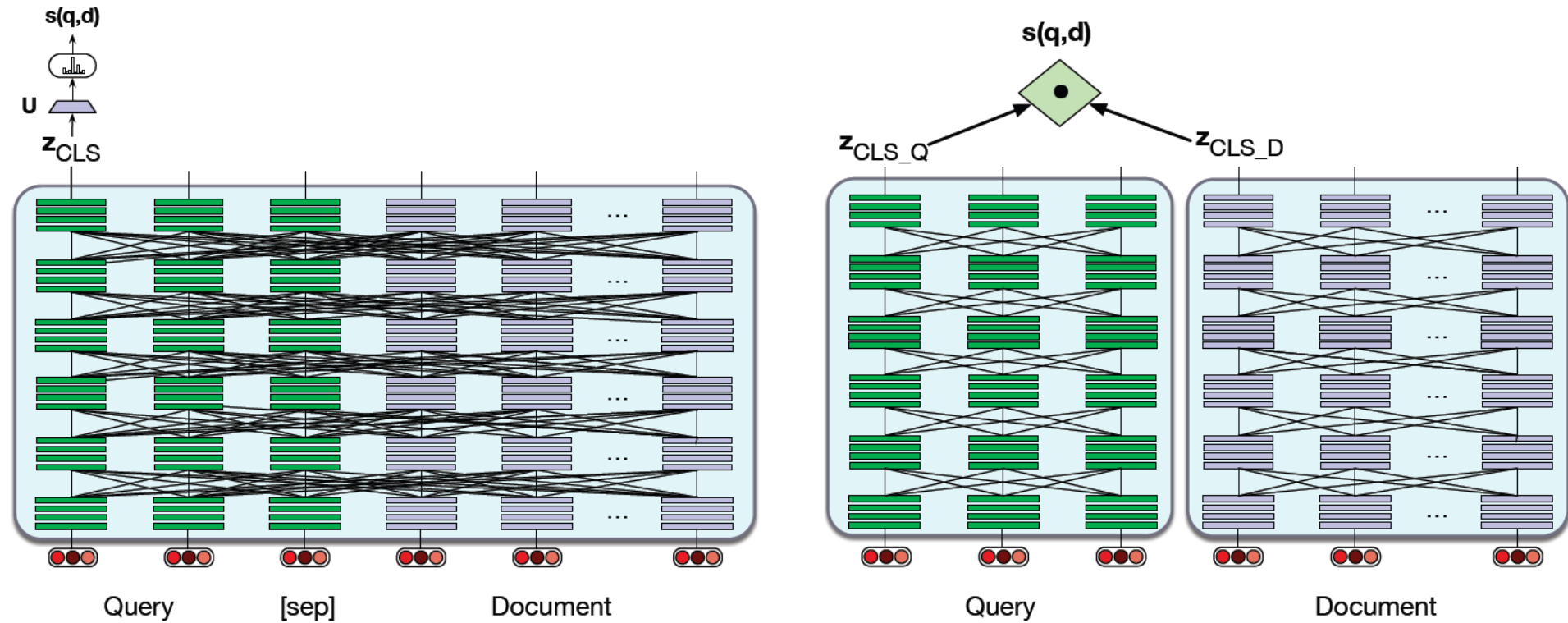
Dense retrieval

- words, sentences, or documents are represented by dense representations (embeddings)
- queries and texts are mapped to dense vectors and the computations use these vectors
- eliminates the problem of exact word overlap of sparse methods

Dense retrieval

- static embeddings (e.g., word2vec, GloVe) can be used, e.g., for query expansion
 - don't capture the context
 - good for simple tasks
- contextual embeddings (e.g., BERT)
 - capture the context
 - are computationally more expensive

Dense retrieval



Dense retrieval

- Q and D can be processed by a single encoder (cross-encoder)
 - finetuned to produce a good relevance score
 - more computations, better accuracy
 - good for smaller datasets, specialized domains, complex queries, where accuracy is important
- Q and D processes by separate encoders (bi-encoder)
 - dot product as the relevance score
 - less computations, less accurate
 - document representations can be precomputed

Hybrid IR

- NN or embedding based IR often brings different errors than traditional IR – they can be combined
- traditional methods used for keyword matching, initial search
- dense representations and NN models used to better handle semantics and re-rank documents

Beyond classical IR

- LLMs – understanding complex queries, adapting to new domains, personalizing responses
- multimodal IR
- generative AI

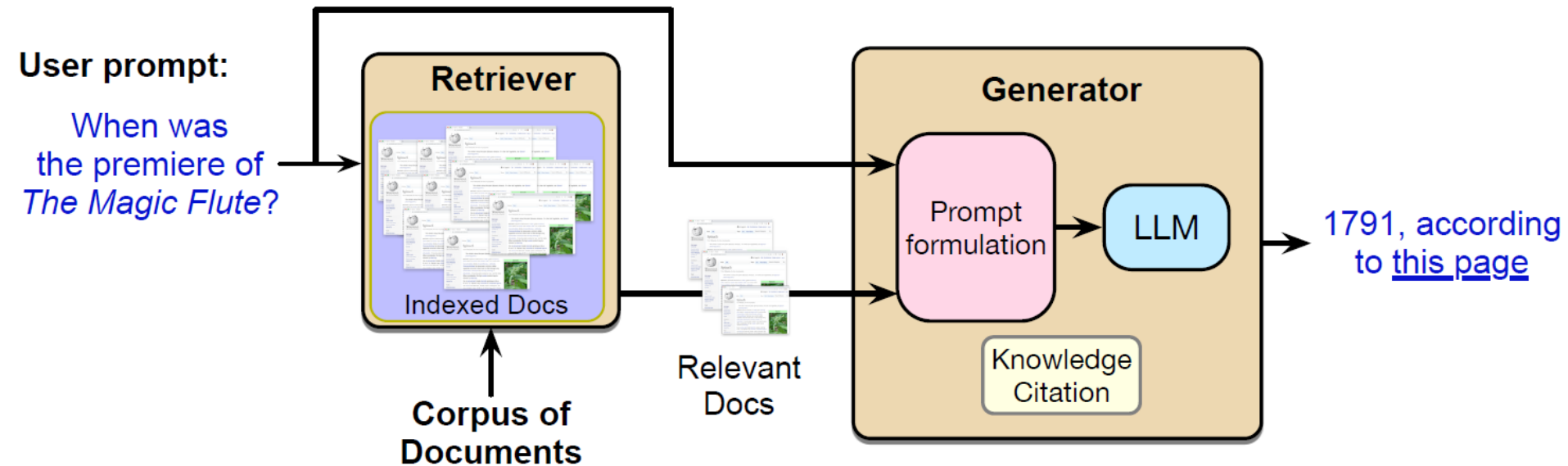
Retrieval-Augmented Generation

- LLMs cannot answer questions about rapidly changing information, personal/sensitive/internal issues, specialized domains (it hallucinates)
- we want the answer of a LLM to be conditioned on the documents that contain the correct answer

Retrieval-Augmented Generation

- retriever
 - retrieves relevant passages of texts from a document collection using IR
 - this allows reaching the information not present in the training data
 - can rely on a vector database that stores vector representations of texts for faster search
- generator
 - creates a prompt containing the query and retrieved passages for an LLM that generates an answer

Retrieval-Augmented Generation



Retrieval-Augmented Generation

- enable improvement of answers without retraining the model
- can provide references to the answer sources
- still might provide wrong answers if wrong documents are retrieved, conflicting information is processed, etc.